HACKATHON JAVA · GENERATION COLOMBIA · COHORTE 15

Domi 🛵

El reto de los domicilios: Java POO desde la terminal

Duración	Equipos	Herramientas	Se entrega
8 horas	4 a 5 personas	IntelliJ IDEA · Java puro (sin Maven ni frameworks)	Repo en GitHub + README + video demo de 3 min

1. Un mensaje de su cliente

> ✉ **De: Valentina Ríos, fundadora de Domi**
>
> Hola, equipo. Domi es una startup bogotana de domicilios. Arrancamos con un local en Chapinero y hoy repartimos a nueve barrios.
>
> Nuestro problema: los pedidos los asignamos **a ojo, por WhatsApp**. A veces mandamos un carro para un pedido de seis cuadras y perdemos dinero. Otras veces mandamos una bici hasta Bosa y el pedido llega frío.
>
> Necesitamos un sistema que **decida solo** qué repartidor le conviene a cada pedido. Para la prueba piloto nos basta una aplicación de terminal que maneje nuestro operador del turno.
>
> Tienen 8 horas. Confío en ustedes. — Valentina

El reto en una frase

> Construir una aplicación de consola en Java que gestione **clientes, repartidores y pedidos** de Domi, y que **decida sola qué repartidor le conviene a cada pedido**.

2. Reglas del hackathon

- **Lean todo el documento antes de escribir la primera línea.** Al final hay un comunicado del cliente que pondrá a prueba su diseño. Si lo conocen desde el inicio, diseñan mejor.
- **Todos los niveles son obligatorios** y se construyen en orden: 🟢 Base → 🟡 Corazón → 🔴 Operación real → 🐉 Jefe final.
- **Un solo repositorio por equipo.** Todos los integrantes deben tener commits.
- **Java puro en IntelliJ.** Los datos viven en memoria mientras el programa está abierto (no hay archivos ni base de datos).
- **La app nunca se debe caer.** Si el operador escribe algo mal, el sistema le avisa y sigue funcionando.
- **Uso de IA:** pueden usarla como **tutor** (para entender un concepto o un error) o como **revisor** (para que revise lo que ustedes ya escribieron). No como copiloto ni agente que escriba el proyecto por ustedes.

> ⚖ **Regla de oro**
>
> En el video, **cualquier integrante debe poder explicar cualquier parte del código**. Si nadie del equipo puede explicar una línea, esa línea no debería estar ahí.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

3. Lo que van a poner en práctica

Concepto	Dónde aparece en Domi
Clases, objetos y encapsulamiento	Cliente, Pedido y los repartidores: atributos privados, getters y setters que validan
Herencia y clase abstracta	`Repartidor` es abstracta. `Bici`, `Moto` y `Carro` heredan de ella
Polimorfismo	Cotizar un pedido: la misma pregunta a todos los repartidores, cada uno responde a su manera
Interfaces	`Calificable`: los repartidores reciben calificaciones y calculan su promedio
ArrayList, HashMap, HashSet	Repartidores, pedidos, clientes por cédula, barrios con cobertura, distancias
Excepciones	Lanzar `IllegalArgumentException` al validar y atraparla con `try/catch` en el menú
Scanner y ciclos	El menú que se repite hasta que el operador elige salir
Git en equipo	Ramas, Pull Requests y un `main` que siempre funciona

4. Cómo organizar el día

Agenda sugerida

Tiempo	Qué hacer	Al terminar
0:00 – 0:45	Leer todo, repartir roles, crear el repo y **dibujar en papel el diagrama de clases**	Diagrama listo
0:45 – 2:30	Nivel 🟢 Base	✅ Checkpoint 1
2:30 – 4:30	Nivel 🟡 Corazón	✅ Checkpoint 2
4:30 – 6:00	Nivel 🔴 Operación real	✅ Checkpoint 3
6:00 – 7:00	Nivel 🪂 Jefe final	Dron funcionando
7:00 – 7:30	Correr todos los casos de prueba (sección 11) y escribir el README	Todo en verde
7:30 – 8:00	Grabar el video demo y entregar	Link en Canvas

Las pausas van según el horario del día. Es una guía: si van adelantados, avancen; si van atrasados, prioricen que cada nivel **funcione** antes de pasar al siguiente.

Roles sugeridos

Rol	Responsabilidad principal
🏗️ Arquitecto/a del modelo	Clases `Cliente`, `Repartidor` y sus hijas, `Pedido` y la interfaz `Calificable`
⚙️ Responsable de reglas	El servicio: cotizar, asignar, cambiar estados, cierre del día
🖥️ Responsable de la terminal	`Main`: el menú, el Scanner, los mensajes y el manejo de errores
🧪 QA (pruebas)	Corre los casos de prueba, busca la forma de "romper" la app y reporta errores
🔀 Integrador/a Git	Crea el repo, revisa y aprueba los Pull Requests, mantiene `main` estable

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

En equipos de 4, QA e Integración Git los asume la misma persona. **Los roles no son muros:** todos programan y todos deben entender el proyecto completo.

Trabajo en equipo con Git

- Una persona crea el repo público y agrega a los demás como colaboradores (Settings → Collaborators).
- Cada tarea se hace en su propia rama, por ejemplo `nivel1-clientes` o `nivel2-cotizacion`, y llega a `main` con un Pull Request.
- `Main.java` **tiene un solo dueño.** Si todos lo editan a la vez, los conflictos están garantizados.
- Commits pequeños y frecuentes, con mensajes que digan qué cambió.

5. Así funciona el negocio de Domi

Antes de programar, entiendan las reglas del negocio. Todo lo que construyan sale de esta sección.

5.1 Zona de cobertura

Domi **solo entrega en estos barrios**. La distancia se mide desde el local de Chapinero.

Barrio	Distancia	Barrio	Distancia	Barrio	Distancia
Chapinero	1,5 km	Usaquén	6,0 km	Suba	12,0 km
Teusaquillo	3,0 km	Cedritos	8,0 km	Kennedy	14,0 km
La Candelaria	4,5 km	Fontibón	11,0 km	Bosa	18,0 km

Las distancias son aproximadas y se usan solo para este reto.

5.2 Tipos de repartidor

Tipo	Tarifa base	Valor por km	Velocidad	Distancia máxima	¿Lleva pedidos grandes?
🚲 Bici	$2.000	$800	15 km/h	5 km	No
🏍️ Moto	$3.000	$1.000	30 km/h	15 km	No
🚚 Carro	$5.000	$1.200	25 km/h	Sin límite	Sí

> 🧮 **Cómo se calcula**
>
> **Costo del domicilio** = tarifa base + (kilómetros × valor por km)
>
> **Tiempo estimado (minutos)** = kilómetros ÷ velocidad × 60
>
> Ejemplo: una moto a Teusaquillo (3 km) cobra 3.000 + 3 × 1.000 = **$6.000** y se demora 3 ÷ 30 × 60 = **6 minutos**.

5.3 Regla de asignación

Cuando llega un pedido, el sistema revisa **todos los repartidores disponibles** que **pueden llevarlo** (por distancia y por tamaño):

- Elige el de **menor costo**.
- Si dos cuestan lo mismo, elige el **más rápido**.
- El repartidor asignado queda **no disponible** hasta que el pedido se entregue o se cancele.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

- Si ningún repartidor puede llevar el pedido, el pedido **no se crea** y el sistema lo informa.

5.4 Estados de un pedido

Estado	Cuándo	Puede pasar a
EN_CAMINO	Apenas se crea y se asigna el pedido	ENTREGADO o CANCELADO
ENTREGADO	El operador confirma la entrega	Ninguno (estado final)
CANCELADO	El cliente se arrepiente antes de recibir	Ninguno (estado final)

6. La aplicación que van a construir

6.1 El menú

Así se puede ver la terminal. No tiene que ser idéntica, pero **sí debe tener todas las opciones**.

```
═══════════════ DOMI · OPERADOR ═══════════════
🌧️  MODO LLUVIA ACTIVO          (solo si está activado)
 CLIENTES      1. Registrar     2. Listar
               3. Editar        4. Eliminar
 REPARTIDORES  5. Registrar     6. Listar
               7. Cambiar disponibilidad
 PEDIDOS       8. Crear pedido (cotiza y asigna)
               9. Entregar     10. Cancelar
              11. Listar pedidos
 OPERACIÓN    12. Activar / desactivar lluvia
              13. Cierre del día
  0. Salir
═══════════════════════════════════════════════
```

6.2 Arquitectura sugerida

Organicen el proyecto en tres paquetes. Cada uno tiene una responsabilidad distinta:

Paquete	Qué contiene	Su trabajo
`modelo`	`Cliente`, `Repartidor` (abstracta), `Bici`, `Moto`, `Carro`, `Pedido` y la interfaz `Calificable`	Representar el negocio. Cada clase se valida a sí misma
`servicio`	`DomiService`	Guardar las colecciones y aplicar las reglas del negocio. **No lee del teclado ni imprime menús**
`app`	`Main`	Mostrar el menú, leer lo que escribe el operador, llamar al servicio y mostrar resultados

> 💡 **Para no perderse**
>
> `Main` **pregunta y muestra. El servicio decide. El modelo se valida a sí mismo.** Si una regla del negocio termina dentro de `Main`, está en el lugar equivocado.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

6.3 Dónde guardar los datos

Qué se guarda	Colección	Pregunta para el README
Clientes, buscados por cédula	`HashMap` (cédula → cliente)	¿Por qué un mapa y no una lista?
Barrios con cobertura	`HashSet` de textos	¿Qué ventaja tiene que no se repitan?
Distancia de cada barrio	`HashMap` (barrio → km)	¿Por qué el operador nunca escribe los km?
Repartidores	`ArrayList`	¿Por qué aquí sí sirve una lista?
Pedidos	`ArrayList`	¿Cómo encuentran un pedido por su número?

6.4 Datos de arranque

Para no perder tiempo digitando, **al iniciar el programa el servicio debe cargar estos datos automáticamente**:

- Los **nueve barrios** de la sección 5.1, con sus distancias.
- Tres repartidores: **R-01 Camila Torres** (Bici), **R-02 Andrés Mejía** (Moto) y **R-03 Jorge Pardo** (Carro).
- Dos clientes: **Laura Gómez**, cédula 1020304050, tel. 3001234567, Teusaquillo; y **Mateo Rincón**, cédula 1030405060, tel. 3109876543, Usaquén.

7. Nivel 🟢 Base: la agenda de Domi

Meta: un menú que funciona para gestionar clientes y repartidores, y que no se cae con nada.

Paso 1 · Proyecto y repositorio

Creen el proyecto en IntelliJ con los tres paquetes, súbanlo al repo y verifiquen que **todos** pueden clonarlo y abrirlo.

Paso 2 · La clase Cliente

Atributos: **cédula, nombre, teléfono, barrio y dirección**. Los setters validan y, si algo está mal, lanzan `IllegalArgumentException` con un mensaje claro para el operador:

- Cédula, nombre y dirección: no pueden estar vacíos. Quiten los espacios sobrantes.
- Teléfono: solo números, entre 7 y 10 dígitos.

> 💡 **Pistas**
>
> - Para revisar que el teléfono solo tenga números, recorran el texto letra por letra. Busquen en la documentación `Character.isDigit`.
> - ¿Quién debe validar que el barrio tenga cobertura? El cliente no conoce la lista de barrios… pero el servicio sí.

Paso 3 · Los repartidores

Creen la clase abstracta `Repartidor` con **código, nombre y disponible** (sí o no), y sus tres hijas: `Bici`, `Moto` y `Carro`. Por ahora sin cálculos: esos llegan en el nivel 2.

Pregunta para el equipo: ¿por qué tiene sentido que nadie pueda crear "un repartidor" a secas?

Paso 4 · El servicio y los datos de arranque

Creen `DomiService` con sus colecciones (sección 6.3) y carguen los datos de arranque (sección 6.4) apenas se crea el servicio.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

Paso 5 · Las opciones 1 a 7 del menú

Opción	Qué debe hacer
1. Registrar cliente	Pide los datos y lo guarda. **No acepta** una cédula repetida ni un barrio sin cobertura
2. Listar clientes	Muestra todos los clientes. Si no hay, lo dice
3. Editar cliente	Busca por cédula y permite cambiar teléfono, barrio y dirección. La cédula no se edita: ¿por qué?
4. Eliminar cliente	Busca por cédula y lo elimina (en el nivel 2 se agrega una condición)
5. Registrar repartidor	Pide código, nombre y tipo. El código no se puede repetir
6. Listar repartidores	Muestra código, nombre, tipo y si está disponible
7. Cambiar disponibilidad	Busca por código y cambia entre disponible y no disponible

Paso 6 · Que no se caiga

Envuelvan cada opción del menú en un `try/catch`. Cuando el modelo o el servicio lancen una excepción, el menú muestra el mensaje y vuelve a empezar.

> 💡 **Pista**
>
> ¿Recuerdan la trampa del buffer del Scanner? Lo más seguro es leer **todo** como texto con `nextLine()` y convertir a número cuando haga falta. Si el operador escribe letras donde va un número, ¿qué excepción sale y cómo la atrapan?

✅ Checkpoint 1

- ☐ El programa arranca con los datos de arranque cargados.
- ☐ Las opciones 1 a 7 funcionan.
- ☐ Una cédula repetida, un teléfono con letras o un barrio sin cobertura muestran un mensaje y **no** tumban el programa.
- ☐ Los casos de prueba 1 y 2 de la sección 11 dan el resultado esperado.
- ☐ Hay commit en `main` con todo esto funcionando.

8. Nivel 🟡 Corazón: cotizar y asignar

Meta: que el sistema decida solo qué repartidor lleva cada pedido. **Aquí está el reto real del hackathon.**

Paso 1 · Enseñarles a los repartidores a cotizar

En `Repartidor` declaren tres comportamientos **abstractos**. Cada hija los implementa con su fila de la tabla 5.2:

- **Calcular costo** para una cantidad de km.
- **Calcular tiempo** para una cantidad de km.
- **Puede llevar** un pedido, según los km y si es grande.

Pregunta para el equipo: ¿por qué estos métodos no se pueden escribir en `Repartidor` directamente?

Paso 2 · La clase Pedido

Atributos: **número** (consecutivo automático que empieza en 1), **cliente, descripción, si es grande, km, repartidor asignado, costo, tiempo y estado**.

- Los km **no los escribe el operador**: salen del barrio del cliente.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

- El estado es un texto que **solo** acepta EN_CAMINO, ENTREGADO o CANCELADO. Cualquier otro valor lanza una excepción.

> 💡 **Pista**
>
> Para validar el estado, ¿qué colección les dice rápidamente si un texto está entre los permitidos?

Paso 3 · Crear pedido (opción 8)

El flujo completo:

- Pedir la cédula del cliente y verificar que exista.
- Pedir la descripción y si el pedido es grande.
- Obtener los km según el barrio del cliente.
- Recorrer los repartidores disponibles y, para cada uno que **puede llevarlo**, mostrar su cotización.
- Elegir el mejor según la regla 5.3, crear el pedido en EN_CAMINO y dejar al repartidor no disponible.

Así se podría ver:

```
Cotizando pedido para Laura Gómez · Teusaquillo (3,0 km)
  R-01 Camila Torres  (Bici)   $4.400   12 min
  R-02 Andrés Mejía   (Moto)   $6.000    6 min
  R-03 Jorge Pardo    (Carro)  $8.600    7 min
✔ Pedido #1 asignado a Camila Torres (Bici) · EN_CAMINO
```

> 🚩 **Señal de alarma**
>
> Si dentro del servicio escriben algo como "si es bici cobra esto, si es moto cobra aquello"… **deténganse**. Cada objeto ya sabe calcular su costo. El servicio solo le pregunta a cada uno. Eso es polimorfismo, y el jefe final lo va a poner a prueba.

> 💡 **Pista**
>
> ¿Cómo encuentran el número menor de una lista? Guardan el primero como "el mejor hasta ahora" y lo comparan con cada uno de los demás. Aquí es igual, pero comparando cotizaciones.

Paso 4 · Entregar, cancelar y listar

Opción	Qué debe hacer
9. Entregar	Busca el pedido por número. Solo si está EN_CAMINO: pasa a ENTREGADO y el repartidor vuelve a estar disponible
10. Cancelar	Solo si está EN_CAMINO: pasa a CANCELADO y el repartidor vuelve a estar disponible
11. Listar pedidos	Muestra número, cliente, repartidor, costo, tiempo y estado de cada pedido

Y una regla pendiente del nivel 1: **no se puede eliminar un cliente que tenga un pedido EN_CAMINO.**

✅ Checkpoint 2

- ☐ Los casos de prueba 3 al 9 de la sección 11 dan el resultado esperado.
- ☐ No se puede entregar ni cancelar un pedido que ya está ENTREGADO o CANCELADO.
- ☐ El servicio no pregunta "¿de qué tipo eres?" a ningún repartidor.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

9. Nivel 🔴 Operación real

Meta: que Domi funcione como un negocio de verdad, con calificaciones, lluvia y cuentas claras al final del día.

A · Calificaciones

Creen la interfaz `Calificable` con dos comportamientos: **recibir una calificación** (de 1 a 5; otro valor lanza excepción) y **obtener el promedio** (0 si aún no tiene calificaciones).

- `Repartidor` implementa `Calificable`. ¿Qué pasa entonces con `Bici`, `Moto` y `Carro`?
- Al **entregar** un pedido (opción 9), el sistema pide la calificación del cliente.
- Al **listar repartidores** (opción 6), se muestra su promedio.

> 💡 **Pista**
>
> Pueden guardar cada calificación en una lista o llevar solo la suma y la cantidad. Las dos son válidas: decidan y expliquen por qué en el README.

B · Modo lluvia (opción 12)

El operador lo activa o desactiva. Mientras está activo:

- El menú muestra un aviso de **MODO LLUVIA ACTIVO**.
- Las **bicis no pueden llevar pedidos**.
- **Todos los costos suben un 20 %.**

Pregunta de diseño: ¿quién debería saber que está lloviendo y dónde va esta regla? Hay varias formas correctas. Elijan una y explíquenla en el README.

C · Cierre del día (opción 13)

Un reporte que muestra:

- Total de pedidos y **cuántos hay en cada estado**.
- **Ingresos del día**: solo cuentan los pedidos ENTREGADOS.
- **Ingresos por tipo de repartidor** (cuánto generaron las bicis, las motos y los carros).
- El **repartidor estrella**: el de mejor promedio entre los que tienen al menos una calificación.

> 💡 **Pista**
>
> Para contar pedidos por estado o sumar ingresos por tipo, piensen en un mapa donde la llave es la categoría y el valor es el acumulado.

✅ Checkpoint 3

- ☐ El caso de prueba 10 de la sección 11 da el resultado esperado.
- ☐ Una calificación de 0 o de 6 se rechaza con un mensaje.
- ☐ El cierre del día cuadra con los pedidos que hicieron en la prueba.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

10. 📢 Comunicado urgente del cliente

> 🐲 **Nivel Jefe final: Domi lanza drones**
>
> "Equipo, ¡noticia enorme! Cerramos un convenio y desde mañana operamos con **drones de reparto**. Necesito que el sistema los soporte hoy mismo." — Valentina
>
> Datos del **Dron**: tarifa base **$4.000**, **$600** por km, velocidad **50 km/h**, distancia máxima **8 km**. **No lleva pedidos grandes** y **no vuela con lluvia**.

La prueba

Agreguen el dron **sin modificar la lógica de cotización y asignación del servicio**.

- **Permitido:** crear la clase `Dron` y agregar la opción "Dron" al registrar un repartidor.
- **Señal de alerta:** si tienen que tocar el método que cotiza y asigna, su diseño tiene una grieta. Arréglenla: ese es el verdadero reto.

Verifiquen con los casos 11 y 12 de la sección 11.

En el README respondan: **¿cuántos archivos tuvieron que tocar para agregar el dron y por qué?**

11. Casos de prueba de aceptación

Así verifica Domi que el sistema funciona. **Cada caso parte de los datos de arranque, con todos los repartidores disponibles.** Los tiempos se pueden mostrar redondeados.

#	Nivel	Situación	Resultado esperado
1	🟢	Registrar un cliente con teléfono 300ABC1234	Mensaje de error; el cliente no se registra
2	🟢	Registrar un cliente en el barrio Soacha	Mensaje de error: sin cobertura
3	🟡	Pedido normal a Chapinero (1,5 km)	Bici · $3.200 · 6 min
4	🟡	Pedido normal a Teusaquillo (3 km)	Bici · $4.400 · 12 min
5	🟡	Pedido normal a Usaquén (6 km)	Moto · $9.000 · 12 min (la bici no llega tan lejos)
6	🟡	Pedido **grande** a Teusaquillo	Carro · $8.600 · 7,2 min
7	🟡	Pedido normal a Bosa (18 km)	Carro · $26.600 · 43,2 min (la moto no llega tan lejos)
8	🟡	Pedido a Teusaquillo con la Bici **no disponible**	Moto · $6.000 · 6 min
9	🟡	Cancelar un pedido que ya fue ENTREGADO	Mensaje de error; el pedido sigue ENTREGADO
10	🔴	Pedido a Teusaquillo con **lluvia**	Moto · $7.200 · 6 min (sin bici y con +20 %)
11	🐲	Con un Dron registrado: pedido normal a Usaquén (6 km)	Dron · $7.600 · 7,2 min
12	🐲	Con un Dron registrado y **lluvia**: pedido a Chapinero	Moto · $5.400 · 3 min (ni bici ni dron)

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

12. Qué se entrega

Entregable	Qué debe tener
📁 Repositorio público en GitHub	El proyecto completo, con commits de todos los integrantes
📄 README.md	Nombre del equipo · integrantes y su rol · cómo ejecutar el proyecto en IntelliJ · las respuestas a las preguntas de la sección 6.3 · dónde quedó la regla de la lluvia y por qué · la respuesta del jefe final · la reflexión de cierre (sección 15)
🎬 Video demo (máximo 3 min)	Link en el README (YouTube oculto o Google Drive). Ver el guion abajo
🔗 Canvas	El link del repositorio, subido antes del cierre

🎬 Guion sugerido para el video

- **15 s:** quiénes son y qué construyeron.
- Registrar un cliente con un error a propósito y mostrar que la app no se cae.
- Crear un pedido y mostrar la cotización de todos los repartidores.
- Activar la lluvia y crear otro pedido.
- Registrar un dron y mostrar que el sistema lo usa.
- Entregar con calificación y cerrar el día.

13. Cómo se evalúa

Criterio	Qué se revisa	Puntos
🟢 Nivel Base	Opciones 1 a 7, validaciones, datos de arranque, la app no se cae	20
🟡 Nivel Corazón	Cotización polimórfica, regla de asignación, estados, entregar y cancelar	25
🔴 Operación real	Calificaciones con interfaz, modo lluvia, cierre del día	20
🐉 Jefe final	El dron funciona sin tocar la lógica de asignación	10
Calidad del código	Encapsulamiento, sin reglas de negocio en `Main`, nombres claros, sin código repetido	15
README y video	Completos, claros, y cualquier integrante puede explicar el código	10
	Total	**100**

⭐ Puntos extra (hasta +5)

- Investigar qué es un **enum** en Java y usarlo para los estados del pedido o los tipos de repartidor.
- Mostrar el historial de pedidos de un cliente.
- Cualquier mejora que Valentina agradecería. Explíquenla en el README.

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack

14. Si se atascan

Problema	Por dónde empezar
"El Scanner se salta una pregunta"	Es la trampa del buffer. Lean todo con `nextLine()` y conviertan cuando haga falta
"El programa se cierra cuando escribo algo mal"	Falta un `try/catch` alrededor de esa opción del menú
"No sabemos dónde poner una regla"	Pregúntense: ¿quién tiene la información para decidir? Ahí va la regla
"Tenemos un conflicto en Git"	Abran el archivo, busquen las marcas del conflicto, decidan juntos qué se queda y hagan commit
"Llevamos 20 minutos en el mismo error"	Lean el mensaje del error de arriba hacia abajo, pregúntenle a la IA como tutor o llamen a la profe

15. Reflexión de cierre

> 🤔 **Para responder en el README**
>
> Cierren el programa y vuelvan a abrirlo. **¿Qué pasó con los pedidos del día?**
>
> Si Domi abre una segunda sede con otro operador, **¿cómo compartirían los clientes y los pedidos entre las dos?**
>
> Guarden su respuesta: la retomamos en la próxima clase de bases de datos.

¡Mucho éxito, equipo! Valentina y Domi cuentan con ustedes. 🛵

© 2026 Ana Alvarado · Educadora Tech & Desarrolladora Full Stack · Todos los derechos reservados · linkedin.com/in/ana-alvarado-instructora-full-stack